

OpsPilot – An AI-Assisted Internal Developer Platform

Mini Project Report

Submitted by **JESTIN SHAJI** *Reg. No.:* **AJC25MCA-2025**

In Partial fulfilment for the Award of the Degree of **MASTER OF COMPUTER APPLICATIONS (MCA)** **APJ ABDUL KALAM**
TECHNOLOGICAL UNIVERSITY

AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS KANJIRAPPALLY [Affiliated to APJ Abdul Kalam Technological
University, Kerala. Approved by AICTE, Accredited by NAAC. Koovappally, Kanjirappally, Kottayam, Kerala 686518]

2026-2027

DEPARTMENT OF COMPUTER APPLICATIONS

AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS KANJIRAPPALLY

<h2>CERTIFICATE</h2>

This is to certify that the Project report, “**OpsPilot**” is the bona fide work of **JESTIN SHAJI (Regno: AJC25MCA-2025)** carried out in partial fulfilment of the requirements for the award of the Degree of **Master of Computer Applications** at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, Affiliated to **APJ Abdul Kalam Technological University**. The project was undertaken during the period from **July 01, 2026 to October, 2026**.

Ms. Susmin Mariam Chacko

Ms. Meera Rose Mathew *Internal Guide*

Coordinator

Dr. Bijimol T.K.

Head of the Department

DECLARATION

I hereby declare that the project report “**OpsPilot**” is a bona fide work done at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, Affiliated to **APJ Abdul Kalam Technological University**, towards the partial fulfilment of the requirements for the award of the **Master of Computer Applications (MCA)** during the period from **July 01, 2026 to October, 2026**.

Date:

Jestin Shaji KANJIRAPPALLY

Reg: AJC25MCA-2025

ACKNOWLEDGEMENT

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully.

I wish to express my sincere gratitude to our Director (Administration) **Rev.Fr. Dr. Roy Abraham Pazhayaparampil** and Principal **Dr. Lillykutty Jacob** for providing good faculty for guidance.

I owe a great depth of gratitude towards our Head of the Department **Dr. Bijimol T.K.** for helping us. I extend my wholehearted thanks to the project coordinator **Ms. Meera Rose Mathew** for her valuable suggestions and for overwhelming concern and guidance from the beginning to the end of the project. I would also express sincere gratitude to my guide **Ms. Susmin Mariam Chacko** for her inspiration and helping hand.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

Jestin Shaji

ABSTRACT

OpsPilot is an AI-assisted Internal Developer Platform (IDP) designed to abstract away the complexities of modern cloud-native infrastructure, enabling developers to build, deploy, and manage applications seamlessly. In today's software engineering landscape, developers often face cognitive overload due to the vast array of DevOps tools, CI/CD pipelines, Kubernetes configurations, and observability stacks required to ship code. OpsPilot centralizes these operations into a single, unified interface.

Built using a microservices architecture with Spring Boot 3 on the backend and React/TypeScript on the frontend, OpsPilot offers automated CI/CD pipeline execution, one-click Kubernetes deployments, real-time log aggregation, and intelligent incident management. The platform features role-based access control, secure OAuth2 integration, and real-time monitoring via Prometheus and Grafana integrations. By bridging the gap between development and operations, OpsPilot significantly reduces time-to-market, minimizes infrastructure mismanagement, and empowers development teams to focus purely on writing code.

CONTENTS

1. **INTRODUCTION** 1.1 Project Overview 1.2 Project Specification
2. **SYSTEM STUDY** 2.1 Introduction 2.2 Existing System 2.3 Drawbacks of Existing System 2.4 Proposed System 2.5 Advantages of Proposed System
3. **REQUIREMENT ANALYSIS** 3.1 Feasibility Study 3.2 System Specification
4. **SYSTEM DESIGN** 4.1 UML Diagrams 4.2 Database Design
5. **SYSTEM TESTING** 5.1 Introduction 5.2 Test Plan
6. **IMPLEMENTATION** 6.1 Introduction 6.2 Implementation Procedure
7. **CONCLUSION & FUTURE SCOPE** 7.1 Conclusion 7.2 Future Scope
8. **BIBLIOGRAPHY**
9. **APPENDIX** 9.1 Sample Code 9.2 Screen Shots

List of Abbreviations

- **API:** Application Programming Interface
- **CI/CD:** Continuous Integration / Continuous Deployment
- **IDP:** Internal Developer Platform
- **JSON:** JavaScript Object Notation
- **JWT:** JSON Web Token
- **K8S:** Kubernetes
- **OIDC:** OpenID Connect
- **RDBMS:** Relational Database Management System
- **REST:** Representational State Transfer
- **UI/UX:** User Interface / User Experience
- **VCS:** Version Control System

CHAPTER 1: INTRODUCTION

1.1 PROJECT OVERVIEW

OpsPilot is an enterprise-grade Internal Developer Platform (IDP) created to streamline the software delivery lifecycle. Organizations today rely heavily on microservices, Kubernetes, Docker, and various CI/CD tools. This fragmentation creates a steep learning curve for developers. OpsPilot acts as a centralized control plane, abstracting these DevOps complexities.

Developers can link their Git repositories, and OpsPilot automatically provisions the necessary CI/CD pipelines, builds Docker images, and deploys them to Kubernetes clusters. The system integrates observability tools, providing real-time logs and incident tracking directly within the developer's dashboard.

1.2 PROJECT SPECIFICATION

OpsPilot consists of several core microservices (Auth Service, Core Service, Observability Service, API Gateway) and a React frontend.

Key Functionalities:

1. **User Authentication:** OAuth2/OIDC integration (Google Login), JWT-based session management, and RBAC (Admin, DevOps, Developer).
2. **Project Management:** Connect GitHub repositories, manage environment variables, and configure build contexts.
3. **CI/CD Automation:** Automated building, testing, and pushing of Docker containers to registries.
4. **Kubernetes Orchestration:** Deploy applications to K8s, manage pods, scale replicas, and handle rollbacks.
5. **Observability:** Centralized log explorer, real-time log streaming via SSE, metrics dashboard, and incident tracking.

CHAPTER 2: SYSTEM STUDY

2.1 INTRODUCTION

A system study for an Internal Developer Platform involves analyzing how developers and operations teams interact with existing tools and identifying bottlenecks in the software delivery process.

2.2 EXISTING SYSTEM

In most organizations without an IDP, developers must manually configure Jenkins/GitLab CI pipelines, write complex Kubernetes YAML manifests, manage Dockerfiles, and search for logs across disparate systems (Kibana, CloudWatch). This is often referred to as "Shadow Operations," where developers spend a significant portion of their time doing DevOps work instead of coding.

2.3 DRAWBACKS OF EXISTING SYSTEM

- **High Cognitive Load:** Developers must learn multiple DevOps tools.
- **Inconsistent Environments:** "It works on my machine" syndrome due to lack of standardized deployment templates.
- **Slow Time-to-Market:** Manual approvals and complex pipeline configurations delay releases.
- **Scattered Observability:** Troubleshooting requires jumping between AWS consoles, Grafana, and terminal windows.

2.4 PROPOSED SYSTEM

OpsPilot provides a "Golden Path" for software delivery. It offers a self-service portal where developers can deploy code with a few clicks. The platform abstracts Kubernetes concepts into simple "Deployments" and "Services". It aggregates logs from all containers into a single searchable UI.

2.5 ADVANTAGES OF PROPOSED SYSTEM

- **Developer Autonomy:** Developers can deploy and monitor their own apps without waiting for DevOps teams.
- **Standardization:** Enforces organizational best practices through standardized CI/CD pipelines.
- **Centralized Observability:** All logs, metrics, and incidents are tracked in one place.
- **Security:** Integrated secret management and strict role-based access control.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDY

The feasibility study evaluates the technical and economic viability of OpsPilot.

- **Technical Feasibility:** The system is highly feasible as it leverages industry-standard open-source tools: Spring Boot for robust backend microservices, PostgreSQL for relational data, and Docker/Kubernetes for orchestration.
- **Economic Feasibility:** By utilizing open-source infrastructure (K3s, Prometheus), the operational costs are kept extremely low. The platform drastically improves developer productivity, ensuring a high Return on Investment (ROI).

3.2 SYSTEM SPECIFICATION

Hardware Requirements:

- Processor: Intel Core i5 / AMD Ryzen 5 or higher
- RAM: 8 GB (16 GB recommended for running local Kubernetes/K3s)
- Storage: 256 GB SSD

Software Requirements:

- Frontend: React 18, TypeScript, TailwindCSS, Vite
- Backend: Java 17, Spring Boot 3.x, Spring Cloud Gateway
- Database: PostgreSQL 15, Flyway Migration
- Infrastructure: Docker, Kubernetes (K3s/Minikube)
- Build Tools: Maven / npm

CHAPTER 4: SYSTEM DESIGN

4.1 UML DIAGRAMS

Use Case Diagram: Actors: Developer, Admin. Use Cases: Login via Google, Connect Repository, Trigger Pipeline, View Logs, Manage Deployments, Resolve Incidents.

4.2 DATABASE DESIGN

OpsPilot utilizes a relational database (PostgreSQL) with strict normalization (3NF) to maintain data integrity.

1. **tbl_users**

- id (PK), email, name, avatar_url, role_id, created_at

2. **tbl_projects**

- id (PK), project_name, github_repo_name, repository_url, owner_id (FK), status

3. **tbl_deployments**

- id (PK), project_id (FK), version, environment, status, deployed_at

4. **tbl_pipeline_runs**

- run_id (PK), project_id (FK), commit_sha, branch, status, exit_code, build_logs

5. **tbl_logs**

- log_id (PK), project_id (FK), deployment_id (FK), source_service, log_level, message, timestamp

6. **tbl_incidents**

- id (PK), project_id (FK), title, description, severity, status, created_at

CHAPTER 5: SYSTEM TESTING

5.1 INTRODUCTION

System testing ensures all microservices (Auth, Core, Observability) communicate seamlessly through the API Gateway and that the frontend accurately reflects the backend state.

5.2 TEST PLAN

- **Unit Testing:** JUnit 5 and Mockito are used to test Spring Boot business logic (e.g., verifying CI/CD state transitions).
- **Integration Testing:** TestContainers is used to spin up temporary PostgreSQL and Docker instances to test repository persistence and Docker daemon communication.
- **Validation Testing:** Validating JWT token expiry, RBAC restrictions, and form inputs on the frontend.
- **User Acceptance Testing (UAT):** End-to-end testing of the "Commit to Deploy" workflow, ensuring a developer can successfully push code, watch the pipeline run, and view the live logs in the OpsPilot dashboard.

CHAPTER 6: IMPLEMENTATION

6.1 INTRODUCTION

The implementation phase involves deploying the OpsPilot microservices into a production-like environment and configuring the necessary infrastructure (Database, K8s cluster).

6.2 IMPLEMENTATION PROCEDURES

The system is implemented using containerization. A `docker-compose.yml` file is provided to spin up the entire stack locally:

1. Postgres Database
2. Service Registry (Eureka)
3. API Gateway (Port 8080)
4. Auth Service (Port 8081)
5. Core Service (Port 8082)
6. Observability Service (Port 8083)
7. React Frontend (Port 5173)

The Core Service connects to the host machine's Docker daemon via `/var/run/docker.sock` to execute CI/CD builds, and communicates with the Kubernetes cluster using the Fabric8 Kubernetes Client.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

OpsPilot successfully abstracts the complexities of cloud-native deployments. By providing a unified dashboard for CI/CD, Kubernetes management, and observability, it empowers developers to deliver software faster and more reliably. The microservice architecture ensures the platform is scalable, fault-tolerant, and easy to maintain.

7.2 FUTURE SCOPE

- **AI-Driven Incident Resolution:** Integrating LLMs to analyze stack traces and suggest automated fixes for failed pipelines or runtime exceptions.
- **Cost Analytics:** Monitoring cloud resource usage (CPU/Memory) and providing cost-saving recommendations.
- **Advanced Infrastructure as Code (IaC):** Support for Terraform and Crossplane to provision cloud databases and caches directly from the UI.

CHAPTER 8: BIBLIOGRAPHY

1. "Microservices Patterns: With examples in Java" by Chris Richardson.
2. "Kubernetes Up & Running" by Brendan Burns, Joe Beda, Kelsey Hightower.
3. Spring Boot Official Documentation: <https://spring.io/projects/spring-boot>
4. React Documentation: <https://react.dev/>
5. Docker Documentation: <https://docs.docker.com/>

CHAPTER 9: APPENDIX

9.1 SAMPLE CODE

CI/CD Pipeline Execution (Core Service):

```
public void executePipeline(Long runId) {
    PipelineRun run = pipelineRepository.findById(runId).orElseThrow();
    run.setStatus("RUNNING");
    pipelineRepository.save(run);

    try {
        // Build Docker Image
        String buildCmd = String.format("docker build -t %s %s", run.getProject().getGithubRepoName(), run.getRepoUrl());
        Process process = Runtime.getRuntime().exec(buildCmd);
        int exitCode = process.waitFor();

        run.setExitCode(exitCode);
        run.setStatus(exitCode == 0 ? "SUCCESS" : "FAILED");
    } catch (Exception e) {
        run.setStatus("ERROR");
        run.setBuildLogs(e.getMessage());
    }
    pipelineRepository.save(run);
}
```

9.2 SCREENSHOTS

(In the final printed report, include screenshots of the OpsPilot Dashboard, CI/CD Pipeline view, Logs Explorer, and Incident Management page here).